

Trabalho 2 - Árvores Binárias e Análise de complexidade.

Regras Gerais

- Os trabalhos devem ser desenvolvidos em grupos de **até 3 pessoas**. O ideal é que todos os trabalhos futuros sejam desenvolvidos neste mesmo grupo. Caso tenha havido alteração na composição do grupo em relação ao primeiro trabalho, o professor deve ser comunicado imediatamente.
- Os relatórios devem ser entregues em formato PDF através do AVA.
- Apenas um componente de cada grupo deve enviar o relatório, onde deve constar os nomes de todos os componentes do grupo.
- O prazo para entrega do relatório está definido na respectiva atividade no Ava. Não serão aceitas entregas atrasadas.
- Todos os códigos desenvolvidos, bem como o relatório, devem ser disponibilizados em um repositório do github, cujo endereço deve estar especificado no relatório.
- Para a definição da nota serão considerados os códigos desenvolvidos, o relatório entregue e as entrevistas.
- Nas entrevistas os alunos deverão apresentar os códigos desenvolvidos e responder a perguntas referentes às análises apresentadas no relatório. As perguntas podem ser direcionadas a um aluno específico ou ao grupo.
- Alunos de um mesmo grupo podem ter notas diferentes, dependendo da participação nas atividades e do desempenho na entrevista. A falta de participação do aluno nas atividades, bem como o desempenho muito ruim na entrevista pode levar até mesmo a zerar a nota do trabalho.
- Este trabalho valerá 40 pontos (40% da nota do semestre).

Introdução

Os objetivos principais deste trabalho são: (i) implementar bibliotecas de árvores binárias e AVL (ii) aplicar as bibliotecas implementadas em um sistema, (iii) entender como a topologia da árvore pode influenciar na ordem de complexidade de execução dos algoritmos e (iv) pesquisar estruturas de árvores binárias disponíveis na biblioteca padrão de Java.

Um dos requisitos do relatório a ser elaborado (etapa 3 do trabalho) é apresentar uma planilha descrevendo a atuação de cada componente do grupo no desenvolvimento do trabalho. Assim, lembrem-se de manter esta planilha atualizada desde o início do desenvolvimento.

É possível paralelizar o desenvolvimento das várias etapas deste trabalho dividindo tarefas de desenvolvimento entre os componentes do grupo. Assim, sugiro que leiam toda a

especificação e planejem as tarefas antes de iniciarem o desenvolvimento.

1) Desenvolvimento da biblioteca de árvores binárias

O objetivo é implementar uma biblioteca de árvore binária em Java. A estrutura deve ser capaz de armazenar objetos de qualquer tipo, sendo o tipo definido na instanciação da árvore. Além disso, deve ser possível instanciar árvores diferentes para armazenar um mesmo tipo aplicando critérios diferentes de indexação. Por exemplo, deve ser possível criar uma árvore para armazenar alunos indexando por nome e uma outra árvore para armazenar os mesmos alunos mas indexando por matrícula. Para isso devem ser utilizados os recursos de **Generics** e a interface **Comparator**.

IMPORTANTE: A classe de Árvore Binária deve, obrigatoriamente, especializar a classe [ArvoreBinariaBase](https://github.com/victoriocarvalho/tpa_2026_1) disponibilizada no repositório do Github: https://github.com/victoriocarvalho/tpa_2026_1.

Assim garantimos que a biblioteca implementa todos os métodos obrigatórios para este trabalho e que estes métodos seguem as assinaturas necessárias para que sua biblioteca seja utilizada pelos programas criados no trabalho de listas, sem a necessidade de grandes alterações. Ademais, isso garantirá que a biblioteca de árvores tenha um atributo para armazenar o Comparator que será utilizado nos métodos para manipular os nós da árvore e que tenha um construtor que receberá este Comparator.

Ao especializar a classe ArvoreBinariaBase, sua classe precisará também implementar os métodos altura, caminharEmNivel e caminharEmOrdem. Note que o toString da classe retorna o resultado do caminhamento em ordem.

2) App que utilize a biblioteca de Árvore Binária

O aplicativo implementado na etapa 2 do trabalho de listas deve ser adaptado para permitir que o usuário escolha entre instanciar árvores ou listas. Ao iniciar seu programa deve perguntar ao usuário se deseja uma lista ordenada, não ordenada ou uma árvore binária. Como a classe [ArvoreBinariaBase](https://github.com/victoriocarvalho/tpa_2026_1) implementa a interface IColecao, não deve haver nenhuma outra alteração em todo o resto do programa.

3) Primeira seção do relatório: relato sobre o desenvolvimento das bibliotecas e do programas de teste

A entrega deste trabalho se dará por meio de um relatório. A seção 1 deste relatório deve:

- Descrever a atuação de cada um dos componentes do grupo no desenvolvimento do trabalho
- Indicar o endereço do repositório do github onde o projeto estará disponível. O repositório deve ter um README descrevendo a organização do código e como rodá-lo.
- Descrever, em detalhes, como utilizaram LLMs no desenvolvimento do trabalho.

4) Segunda seção do relatório: Análise Matemática de complexidade dos Algoritmos da sua biblioteca

Na seção 2 do relatório você deve desenvolver análises de complexidade dos métodos adicionar, pesquisar, remover e quantidade de elementos da sua biblioteca. As análises devem referenciar os códigos implementados, explicando o raciocínio linha a linha. Assim, é recomendável que se use no relatório imagens dos códigos com as linhas numeradas para facilitar a referência. Para cada método analisado você deve chegar a conclusão sobre qual a ordem de complexidade no pior caso e explicar quando esse pior caso ocorre. Note que no caso das árvores a topologia (formato) da árvore influencia na ordem de complexidade dos métodos. Assim, avalie a ordem de complexidade considerando o pior caso em árvores perfeitamente balanceadas e o pior caso em árvores completamente degeneradas.

5) Terceira seção do relatório: Análise Empírica de complexidade de Algoritmos da sua biblioteca de árvores

Para analisar empiricamente a complexidade dos algoritmos de sua biblioteca, você deve rodar diversas vezes seu programa com arquivos de tamanhos diferentes, assim como foi feito no trabalho anterior. Obviamente desta vez você deve optar por instanciar árvores binárias ao invés de listas.

Para cada tamanho de arquivo (ao menos 4 tamanhos), rode escolhendo árvore binária. A cada execução você deve:

- Anotar o tempo gasto para leitura do arquivo e montagem das árvores;
- Pesquise pelo telefone do elemento que seja o pior caso de busca, ou seja, **uma busca pela folha mais distante da raiz considerando a indexação por telefone**. Anote o tempo gasto na execução do método de busca;
- Pesquise pelo nome do elemento para o qual foi feita a busca por telefone no item anterior e anote o tempo gasto na execução do método de busca;
- Remova, por telefone, o mesmo elemento buscado nos itens anteriores e anote o tempo gasto na execução do método de remoção;

Assim como no trabalho anterior, você deve rodar o programa para arquivos de entrada de ao menos 4 diferentes tamanhos, preferencialmente, dos mesmos tamanhos que você utilizou no trabalho anterior. Porém agora, para cada tamanho, serão dois arquivos: **(i) um arquivo em que os números de telefone estão em ordem crescente (logo gerarão uma árvore degenerada) e (ii) um arquivo em que os registros são ordenados de forma que**

a árvore gerada seja perfeitamente balanceada. Para gerar os arquivos de entrada você pode adaptar os exemplos de programas geradores de arquivos disponíveis em <https://github.com/victoriocarvalho/GeradorArquivos>. Os geradores disponíveis geram arquivos de Alunos, indexados por um identificador numérico. O [gerador de arquivos balanceados](#) gera arquivos que quando lidos para árvores binárias, são inseridos em uma ordem em que a árvore resultante será perfeitamente balanceada (as matrículas são geradas sempre dividindo ao meio o intervalo de matrículas restante até que o intervalo acabe). Já o [gerador de arquivos ordenados](#) gera arquivos em que os identificadores dos objetos são gerados em ordem crescente. Logo, ao serem inseridos em uma árvore geram árvores totalmente degeneradas. Assim, basta adequar os geradores ao domínio tratado no trabalho de vocês.

Então, na seção 3 do relatório você deve:

- escrever uma subseção 3.1 (implementação) na qual vai explicar como gerou os arquivos de entrada e como definiu qual dos elementos do arquivo seria o pior caso caso de busca
- escrever uma subseção 3.2 (análise de complexidade) na qual você exibirá tabelas contendo a quantidade de objetos e os tempos de execução coletados para os diferentes arquivos de entrada utilizados. Em seguida, deve interpretar esses dados coletados discutindo se eles atendem ao esperado considerando as análises matemáticas feitas na seção anterior. Por fim compare os tempos obtidos em árvores degeneradas, em árvores perfeitamente balanceadas e em listas (no trabalho anterior).

6) Desenvolvimento de Árvore AVL

O objetivo desta etapa é acrescentar à sua biblioteca uma classe que implemente árvores AVL. As árvores AVL são árvores binárias que aplicam um algoritmo de balanceamento após a inserção e a remoção de elementos a fim de manter a árvore sempre balanceada.

Sua classe "ArvoreAVL" deve, obrigatoriamente, herdar de "ArvoreBinaria", sobrescrever os métodos "adicionar" e "remover" e implementar métodos para fazer o balanceamento

Inclua a seção 4 no relatório, na qual você deve desenvolver análises de complexidade dos métodos adicionar, e remover de Árvore AVL, considerando o balanceamento.

7) Avaliação do desempenho das Árvores AVL comparando com as Árvores binárias comuns.

Após implementada a classe de Árvore AVL vamos utilizá-la no programa de testes. Para isso altere o programa utilizado no passo 5 deste trabalho de forma que, ao iniciar, ele dê ao usuário as opções de instanciar lista ordenada, lista não ordenada, árvore binária ou árvore AVL.

Para analisar empiricamente a complexidade dos algoritmos de sua biblioteca, você deve rodar seu programa com **os arquivos em que os números de telefone estão em ordem crescente (os mesmos usados no passo 5)**, porém instanciando árvores AVL. Para cada execução faça as mesmas observações de tempo feitas no passo 5 deste trabalho..

Então, na seção 5 do relatório você deve exibir tabelas contendo a quantidade de objetos e os tempos de execução coletados nos testes de árvores AVL. Em seguida, deve interpretar esses dados coletados discutindo se eles atendem ao esperado considerando as análises matemáticas feitas na seção anterior. Por fim compare os tempos obtidos utilizando Árvores AVL com os obtidos utilizando árvores binárias comuns com os mesmos arquivos (**os arquivos em que os números de telefone estão em ordem crescente**).

8) Pesquisa sobre estruturas de árvores binárias disponíveis na biblioteca padrão de Java

Pesquise sobre implementações de árvores binárias disponíveis na biblioteca padrão de Java. Na seção 6 do relatório você deve explicar o funcionamento da(s) implementação(ões) pesquisadas, explicando a estrutura de dados na qual ela se baseia (se usa alguma estratégia de balanceamento, qual?), a estratégia utilizada para comparar elementos e qual a ordem de complexidade de inserções, buscas e remoções nessas estruturas.